

SCIENTIFIC ANALYSIS GROUP (SAG) — DRDO

Quantum Simulation Lab

Technical Project Documentation

Document Type: Comprehensive Technical Reference & Deployment Guide

Version: 1.0.0

Classification: Internal — For Official Use

Date: 30 July 2026

Prepared By: QuantumLab Development Team

Organization: DRDO — Scientific Analysis Group

Live URL: <https://sagqsim.vercel.app/>

CONFIDENTIAL — PRODUCTION GRADE

Table of Contents

- 1. Executive Summary**
- 2. Technology Stack**
 - 2.1 Frontend Technologies
 - 2.2 Backend Technologies
 - 2.3 Quantum Execution Engine
 - 2.4 Third-Party Services & APIs
- 3. System Architecture**
 - 3.1 High-Level Architecture
 - 3.2 Directory Structure
 - 3.3 Data Flow
- 4. Feature Catalogue**
- 5. Database Design**
- 6. API Reference**
- 7. Security Architecture**
- 8. Authentication System**
- 9. Deployment Guide**
 - 9.1 Local Development Setup
 - 9.2 Production Deployment (Vercel + Render)
 - 9.3 Cloudinary Configuration
 - 9.4 Google OAuth Setup
 - 9.5 GNews API Integration
 - 9.6 Redis Cache (Optional)
- 10. Environment Variables Reference**
- 11. Pricing & Cost Analysis**
- 12. Server Migration Guide**
- 13. Scaling & Performance**
- 14. Theming System**
- 15. Quantum Execution Engine**
- 16. Admin Panel**
- 17. Troubleshooting & Known Issues**
- 18. Future Roadmap**
- 19. Appendix**

1. Executive Summary

Quantum Simulation Lab is a production-grade, full-stack web platform developed for the **Scientific Analysis Group (SAG), Defence Research and Development Organisation (DRDO)**. It serves as an interactive research and education platform for quantum algorithm simulation, experimentation, and visualization.

The platform bridges the gap between **Quantum Engineers/Data Scientists** (who write Jupyter Notebooks with Qiskit) and **End Users** (researchers, students, and analysts who interact via a modern web dashboard). Users can learn about quantum computing concepts, configure algorithm parameters through a UI, execute quantum circuits on an IBM Qiskit Aer simulator backend, and visualize results — including quantum circuits, histograms, Bloch spheres, and measurement probabilities — all within the browser.

Key Capabilities

- **Quantum Algorithm Execution:** 12 pre-built quantum algorithms (Deutsch, Grover, Shor, QFT, VQE, etc.) runnable with custom parameters
 - **Python Sandbox:** A full in-browser code editor for writing and executing arbitrary Python/Qiskit code
 - **3D Bloch Sphere Visualization:** Interactive WebGL-based quantum state visualization using Three.js
 - **Circuit Simulator:** Drag-and-drop quantum gate builder with real-time state vector computation
 - **Circuit Challenges:** Gamified puzzle system for building circuits that match target output states
 - **Entanglement Lab:** Bell State creation and measurement visualization
 - **Content Management System:** Full CMS for Documentation, Blogs (with likes/comments), News, and Courses with video lectures
 - **Live News Feed:** Real-time quantum computing news via GNews API integration
 - **Admin Dashboard:** Comprehensive analytics, user management, and content management panel
 - **Google OAuth + JWT Authentication:** Dual authentication with role-based access control
 - **Dark/Light Theme:** Seamless theme toggling via CSS Variables
-

2. Technology Stack

2.1 Frontend Technologies

TECHNOLOGY	VERSION	PURPOSE
React	19.2.7	Core UI framework (latest with concurrent features)
Vite	8.1.1	Build tool & dev server (HMR, ESBuild)
React Router	7.18.1	Client-side routing with nested routes
Tailwind CSS	4.3.2	Utility-first CSS framework
Framer Motion	12.42.2	Page transitions, micro-animations, scroll animations
Three.js / R3F	0.185.1	3D Bloch Sphere and Quantum Atom visualization
@react-three/drei	10.7.7	Three.js helpers (OrbitControls, etc.)
Axios	1.18.1	HTTP client with interceptors for JWT
React Quill (New)	3.8.3	Rich Text Editor for CMS content
KaTeX	0.17.0	LaTeX-style math formula rendering
PrismJS	1.30.0	Code syntax highlighting
Recharts	3.9.2	Analytics charts in Admin Panel
Lenis	1.3.25	Smooth scrolling library
Sonner	2.0.7	Toast notifications
@react-oauth/google	0.13.5	Google Sign-In button component
@dnd-kit	6.3.1	Drag-and-drop for circuit builder & docs reordering
Marked / Turndown	18.0.6 / 7.2.4	Markdown ↔ HTML conversion

2.2 Backend Technologies

TECHNOLOGY	VERSION	PURPOSE
Node.js	18+ (LTS)	Server runtime
Express.js	5.1.0	Web framework (latest v5)
Mongoose	9.7.4	MongoDB ODM
JSON Web Token	9.0.3	JWT authentication
bcryptjs	3.0.3	Password hashing (bcrypt)
Helmet	8.2.0	HTTP security headers
express-rate-limit	8.5.2	DDoS/brute-force protection
express-mongo-sanitize	2.2.0	NoSQL injection prevention
DOMPurify + jsdom	3.4.12 / 29.1.1	Server-side HTML sanitization (XSS prevention)
Multer	2.2.0	File upload handling (multipart/form-data)
Cloudinary SDK	1.41.3	Cloud image storage (production)
google-auth-library	10.9.0	Google OAuth token verification
Redis	6.1.0	Response caching (optional)
CORS	2.8.5	Cross-Origin Resource Sharing
dotenv	17.4.2	Environment variable management

2.3 Quantum Execution Engine (Python)

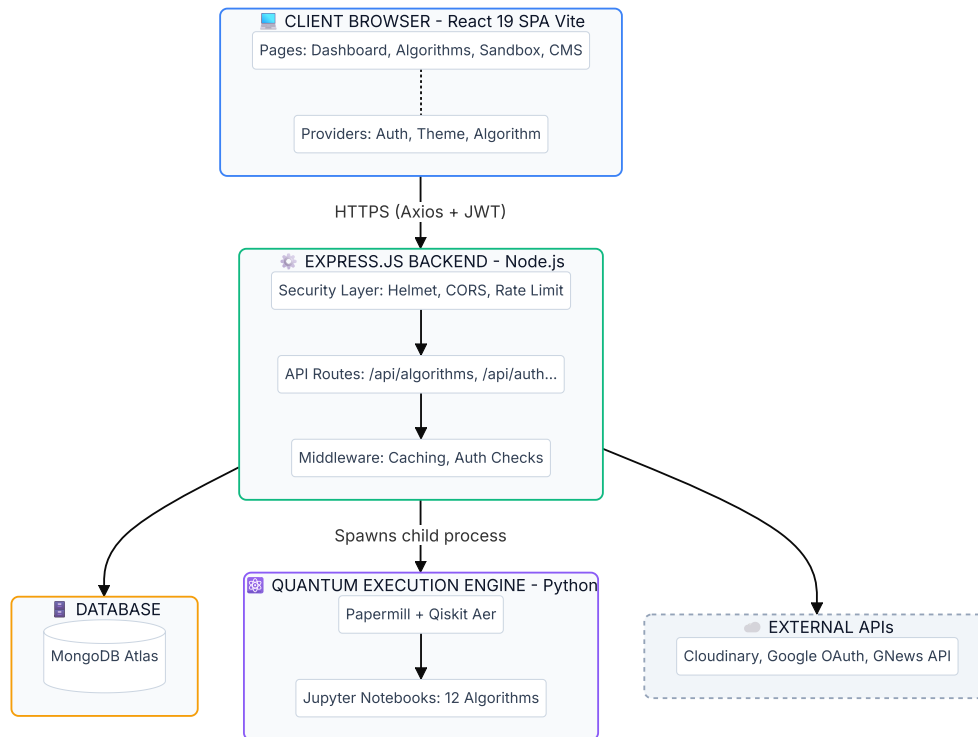
PACKAGE	VERSION	PURPOSE
Qiskit	1.3.1	Quantum circuit construction & execution
Qiskit Aer	0.15.1	High-performance quantum simulator
Papermill	2.6.0	Parameterized Jupyter Notebook execution
Jupyter	1.1.1	Notebook runtime kernel
Matplotlib	3.10.0	Circuit diagram & histogram rendering
NumPy, SciPy, Pandas	latest	Scientific computation
Sympy, NetworkX	latest	Symbolic math & graph algorithms
pylatexenc	latest	LaTeX rendering in circuit diagrams

2.4 Third-Party Services & APIs

SERVICE	PURPOSE	CURRENT TIER
MongoDB Atlas	Cloud database (NoSQL)	Free (M0 Cluster)
Vercel	Frontend hosting & CDN	Free (Hobby)
Render	Backend hosting (Node.js)	Free Tier
Cloudinary	Image CDN & storage	Free (25 credits/month)
Google Cloud Console	OAuth 2.0 (Sign In with Google)	Free
GNews API	Live quantum news feed	Free (100 req/day)
Redis (Optional)	Response caching	Disabled by default

3. System Architecture

3.1 High-Level Architecture



3.2 Complete Directory Structure

```

QuantumSimulationLab/
├── .env # All environment variables (secrets)
├── .env.production # Production API URLs (Vite bakes these at build time)
├── index.html # HTML entry point with SEO meta tags
├── package.json # Frontend dependencies & scripts
├── vite.config.js # Vite config (React, Tailwind, COOP header)
├── vercel.json # Vercel SPA rewrite rules
├── src/ # — Frontend Source Code —
│   ├── App.jsx # Root: Providers → Router → AppInner
│   ├── main.jsx # React DOM entry point
│   ├── index.css # Global styles, Theme System (CSS Vars)
│   ├── components/ # 23 reusable UI components
│   │   ├── Navbar/, Sidebar/, Footer/
│   │   ├── BlochSphereViewer/ # 3D WebGL Bloch Sphere
│   │   ├── QuantumAtom3D/ # 3D quantum atom animation
│   │   ├── QuantumCircuitViewer/ # Circuit image display
│   │   ├── RichTextEditor/ # ReactQuill with 3 modes
│   │   ├── MathHTMLContainer/ # KaTeX math rendering
│   │   ├── ProtectedRoute/ # Auth guard component
│   │   ├── AnimatedPage/ # Framer Motion page transition wrapper
│   │   ├── ThemeToggle/ # Dark/Light mode switch
│   │   └── ... (AlgorithmCard, Button, Loading, etc.)
│   ├── context/
│   │   ├── AuthContext.jsx # JWT auth state + Google OAuth
│   │   ├── AlgorithmContext.jsx # Algorithms list cache
│   │   └── ThemeContext.jsx # Dark/Light theme toggle
  
```

```

|   └─ hooks/
|   |   └─ useAlgorithm.js # Algorithm data fetching hook
|   |   └─ useSmoothScroll.js # Lenis smooth scrolling
|   |   └─ usePageTracking.jsx # Analytics page visit tracker
|   └─ pages/ # 17 page modules
|   |   └─ LandingPage/ # Public marketing page
|   |   └─ LoginPage/ # Login + Registration
|   |   └─ DashboardPage/ # User home (algorithm grid)
|   |   └─ AlgorithmPage/ # Algorithm execution + results
|   |   └─ SandboxPage/ # Python code editor
|   |   └─ CircuitSimulatorPage/ # Drag-and-drop circuit builder
|   |   └─ CircuitChallengesPage/ # Circuit puzzles
|   |   └─ EntanglementLabPage/ # Bell State experiments
|   |   └─ QuantumPlayground/ # Playground hub
|   |   └─ BlochSphereFullScreenPage/
|   |   └─ DocsPage/, BlogsPage/, BlogViewerPage/
|   |   └─ NewsPage/ # DB News + Live GNews
|   |   └─ CoursesPage/ # Video courses catalog + viewer
|   |   └─ AdminPanel/ # 10 admin pages (see Section 16)
|   └─ routes/
|   |   └─ AppRoutes.jsx # All route definitions
|   └─ services/
|   |   └─ api.js # Axios client + all API functions
|   └─ utils/
|   |   └─ constants.js # Algorithm series order, etc.
└─ backend/ # — Backend Source Code —
    └─ server.js # Express entry point (port 8000)
    └─ package.json # Backend dependencies
    └─ requirements.txt # Python dependencies
    └─ config/
    |   └─ db.js # MongoDB connection
    └─ middleware/
    |   └─ auth.js # JWT authenticate + adminOnly
    |   └─ cache.js # Redis caching middleware
    └─ models/ # 10 Mongoose schemas
    |   └─ User.js, Algorithm.js, Blog.js, Doc.js
    |   └─ News.js, Course.js, Challenge.js
    |   └─ PlaygroundItem.js, PageVisit.js
    |   └─ EducationalContent.js
    └─ controllers/ # 10 controller files
    └─ routes/ # 11 route files
    └─ notebooks/ # 12 Qiskit Jupyter Notebooks
    └─ scripts/ # Seed scripts & temp challenge scripts
    └─ uploads/ # Local image uploads (dev mode)
└─ dist/ # Production build output (Vite)

```

3.3 Data Flow

The system follows a decoupled client-server architecture:

1. **Frontend (SPA)** is served as static files from Vercel's CDN. All routes are rewritten to `index.html` via `vercel.json`.
2. **API Calls** are made via Axios to the Express backend on Render. Every authenticated request includes a `Bearer <JWT>` header.
3. **Quantum Execution:** When a user runs an algorithm, the backend spawns a Python child process using `child_process.exec()`, which runs `papermill` to execute the relevant `.ipynb` notebook with user-provided parameters. Output images and text are extracted from the executed notebook and returned as JSON.
4. **Image Uploads:** In development, images are saved to `backend/uploads/`. In production, they are uploaded to Cloudinary via `multer-storage-cloudinary`.

4. Feature Catalogue

#	FEATURE	DESCRIPTION	ACCESS
1	Landing Page	Marketing page with 3D quantum atom animation, smooth scrolling (Lenis), and Framer Motion scroll-triggered animations	Public
2	Authentication	Registration (username/password) + Google OAuth Sign-In. JWT tokens with 7-day expiry. Role-based access (user/admin)	Public
3	Dashboard	Grid of all quantum algorithms with search, algorithm series ordering, and quick-run access	Logged In
4	Algorithm Execution	Configure parameters → Execute via Qiskit Aer → View circuit diagram, measurement histogram, Bloch sphere, and console output	Logged In
5	3D Bloch Sphere	Interactive WebGL Bloch sphere with real-time θ/ϕ angle display. Full-screen mode available	Logged In
6	Python Sandbox	Multi-cell code editor. Execute arbitrary Python code. Install packages via pip. View images and console output	Logged In
7	Circuit Simulator	Drag-and-drop quantum gate builder. Build circuits with H, X, Y, Z, S, T, CX, SWAP gates. Real-time state vector simulation	Logged In
8	Circuit Challenges	Gamified puzzles: build circuits to match target output probability distributions. Verified via Qiskit Statevector	Logged In
9	Entanglement Lab	Create Bell states, apply gates, measure and visualize entanglement	Logged In
10	Quantum Playground	Hub page for all interactive tools (Bloch Sphere, Circuit Sim, Entanglement Lab, Challenges)	Logged In
11	Documentation (CMS)	Hierarchical docs with sections, subsections, drag-and-drop reordering, Rich Text editor with KaTeX math support	Public
12	Blogs (CMS)	Blog posts with full rich text, likes, comments, search, pagination. Blog viewer page with social interactions	Public (interactions require login)
13	News (CMS + GNews)	Internal news articles (admin-created) + Live quantum news from GNews API. Category auto-tagging	Public
14	Courses (CMS)	Video course catalog with lecture playlists. Course viewer page with video player	Public
15	Educational Content	Per-algorithm educational tabs: Description, Problem, Algorithm, Geometric Proof, Algebraic Proof, Code, Applications	Logged In
16	Admin Dashboard	Overview with quick stats and navigation to all admin functions	Admin Only

#	FEATURE	DESCRIPTION	ACCESS
17	Admin Analytics	User demographics (OS, Gender, Age), Page views with time filters, Recharts visualizations	Admin Only
18	User Management	List all users with search/pagination. Promote/demote admin roles	Admin Only
19	Dark/Light Theme	System-wide theme toggle via CSS Variables on the <code><html data-theme></code> attribute	All
20	Page Analytics	Automatic tracking of page visits per path per day, aggregated in admin panel	Background

5. Database Design

The application uses **MongoDB Atlas** (cloud-hosted NoSQL) via the **Mongoose ODM**. The database name is `QuantumSimulationLab`. Below are all 10 collections and their schemas:

5.1 Users Collection

FIELD	TYPE	CONSTRAINTS	NOTES
username	String	required, unique, trim, min 3 chars	Login identifier
name	String	required, trim	Display name
email	String	required, unique, lowercase	Also used for Google OAuth match
dob	Date	required	Used for age analytics
gender	String	enum: Male, Female, Other	Used for gender analytics
phone	String	required	Contact info
os	String	default: "Unknown"	User's OS (captured at registration)
password	String	required, min 6 chars	bcrypt-hashed (salt rounds: 10)
role	String	enum: user, admin (default: user)	RBAC
createdAt / updatedAt	Date	auto (timestamps: true)	Mongoose timestamps

Password Hashing

Passwords are automatically hashed via a Mongoose `pre('save')` hook using `bcryptjs` with a salt factor of 10. Plaintext passwords are never stored.

5.2 Algorithms Collection

FIELD	TYPE	NOTES
id	String (unique, indexed)	Slug identifier (e.g., "grover-search")
name	String	Display name
shortDescription	String	Card description
description	String	Full description
formula	String	KaTeX formula string
timeComplexity / spaceComplexity	String	Big-O notation
reference	String	Academic reference link
category	String	Algorithm series category
parameters	Array of Objects	Each: {name, type, default, min, max, step, placeholder, options}

5.3 Other Collections (Summary)

COLLECTION	KEY FIELDS	PURPOSE
blogs	title, excerpt, category, readTime, author, content, likes[], comments[]	Blog posts with social features
docs	section, subsection, title, sectionOrder, subsectionOrder, order, content	Hierarchical documentation
news	title, source, date, tag, excerpt, content, isFeatured	Internal news articles
courses	title, description, thumbnail, instructor, lectures[{title, videoUrl, duration}]	Video courses with lectures
challenges	title, desc, numQubits, targetState[], targetStr, allowedGates[], order	Circuit challenge puzzles
playgrounditems	icon, title, desc, status, path, order	Playground hub items
educationalcontents	algorithmId, description, problemDescription, algorithm, geometricProof, algebraicProof, code, applications[], limitations[]	Educational content per algorithm
pagevisits	path (indexed), date (indexed), count	Page visit analytics (unique per path+day)

6. API Reference

The backend exposes a RESTful API at `https://quantum-sim-lab-backend.onrender.com/api` . All authenticated endpoints require a `Bearer <JWT>` token in the `Authorization` header.

6.1 Authentication Endpoints

METHOD	ENDPOINT	AUTH	DESCRIPTION
POST	<code>/api/auth/register</code>	Public	Create new user account
POST	<code>/api/auth/login</code>	Public	Login with username/password → returns JWT
POST	<code>/api/auth/google-login</code>	Public	Login/Register via Google ID Token
GET	<code>/api/auth/me</code>	JWT	Get current authenticated user info

6.2 Algorithm Endpoints

METHOD	ENDPOINT	AUTH	DESCRIPTION
GET	<code>/api/algorithms</code>	Public	List all algorithms
GET	<code>/api/algorithms/:id</code>	Public	Get single algorithm by slug
POST	<code>/api/algorithms</code>	Admin	Create new algorithm
PUT	<code>/api/algorithms/:id</code>	Admin	Update algorithm
DELETE	<code>/api/algorithms/:id</code>	Admin	Delete algorithm
POST	<code>/api/run</code>	JWT	Execute algorithm via Papermill

6.3 Content Endpoints (Docs / Blogs / News)

METHOD	ENDPOINT	AUTH	DESCRIPTION
GET	/api/docs	Public	Get all documentation pages
POST	/api/docs	Admin	Create doc page
PUT	/api/docs/reorder	Admin	Bulk reorder docs
PUT	/api/docs/:id	Admin	Update doc
DELETE	/api/docs/:id	Admin	Delete doc
GET	/api/blogs?page=&limit=&search=	Public	Get blogs (paginated)
POST	/api/blogs	Admin	Create blog
POST	/api/blogs/:id/like	JWT	Like/Unlike a blog
POST	/api/blogs/:id/comment	JWT	Add comment to blog
GET	/api/news?page=&limit=&search=	Public	Get internal news (paginated)
GET	/api/gnews/search?q=&page=&max=	Public	Search live GNews API
GET	/api/gnews/top-headlines?topic=	Public	GNews top headlines

6.4 Other Endpoints

METHOD	ENDPOINT	AUTH	DESCRIPTION
POST	/api/sandbox/run	JWT	Execute Python code in sandbox
POST	/api/sandbox/install	JWT	Install Python packages
POST	/api/upload	Admin	Upload image (Cloudinary or local)
POST	/api/challenge/run	JWT	Evaluate circuit challenge
GET	/api/challenges	JWT	List all challenges
GET	/api/courses	Public	List all courses
GET	/api/courses/:id	Public	Get single course with lectures
GET	/api/playground	Public	Get playground items
GET	/api/users	Admin	List all users (paginated)
PUT	/api/users/:id/role	Admin	Update user role
POST	/api/analytics/track-page	Public	Track page visit
GET	/api/analytics/users	Admin	User demographic analytics
GET	/api/analytics/page-views	Admin	Page view analytics

7. Security Architecture

The platform implements multiple layers of security, making it production-ready:

7.1 Security Layers

LAYER	IMPLEMENTATION	DETAILS
HTTP Headers	Helmet.js (v8)	Sets X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security, Content-Security-Policy, etc. Cross-Origin-Resource-Policy set to "cross-origin" for image loading.
CORS	Whitelist-based	Only localhost:5173 , localhost:5174 , 127.0.0.1:5173/5174 , and FRONTEND_URL are allowed. All other origins are rejected with 403.
Rate Limiting	express-rate-limit	Global: 200 req/15 min. Auth endpoints: 15 req/15 min (production), 1000 (development). Returns 429 on excess.
NoSQL Injection	express-mongo-sanitize	Strips \$ and . operators from req.body and req.params before they reach MongoDB queries.
XSS Prevention	DOMPurify (server-side)	All user-submitted HTML (blogs, docs, news) is sanitized through DOMPurify before database storage. Whitelist of safe HTML tags and attributes maintained.
Password Security	bcryptjs (10 rounds)	Passwords are hashed with bcrypt before storage. Plaintext is never persisted. Comparison uses timing-safe methods.
JWT Authentication	jsonwebtoken	256-bit secret key. Token contains user ID, username, and role. 7-day expiry. Transmitted via Bearer token in Authorization header.
Google OAuth	google-auth-library	Server-side verification of Google ID tokens against the registered Client ID. Prevents token forgery.
File Upload Security	Multer + type checking	Only image files (jpeg, jpg, png, gif, webp, svg) allowed. 20 MB size limit. File type validated by both extension and MIME type.
Command Injection	Input sanitization	Sandbox package installs sanitize input to allow only [a-zA-Z0-9_\-\.=\<>] . Challenge circuits use whitelisted gate types only.
Error Handling	Global error handler	In production, internal error details are never leaked to the client. Generic "Internal Server Error" is returned.
COOP Header	Vite config	Cross-Origin-Opener-Policy: same-origin-allow-popups is set to allow Google OAuth popup to communicate back.

 Critical Security Note

The .env file contains all secrets (MongoDB URI, JWT secret, Cloudinary credentials, API keys). This file must **NEVER** be committed to version control or shared publicly. It is included in .gitignore .

8. Authentication System

8.1 Authentication Flow

Traditional Login (Username + Password)

- 1. User submits username & password to `POST /api/auth/login`
- 2. Backend finds user by username, compares password via `bcrypt.compare()`
- 3. If valid, generates JWT containing `{id, username, role}` with 7-day expiry
- 4. Frontend stores JWT in `localStorage` under key `qsl_token`
- 5. Axios interceptor attaches `Bearer <token>` to all subsequent API requests
- 6. On page refresh, `AuthContext` calls `GET /api/auth/me` to re-validate the token

Google OAuth Flow

- 1. User clicks "Sign in with Google" → Google popup opens
- 2. User authenticates with Google → Google returns an ID Token (credential)
- 3. Frontend sends the ID Token to `POST /api/auth/google-login`
- 4. Backend verifies the token using `google-auth-library` against the registered `GOOGLE_CLIENT_ID`
- 5. If user email exists in DB → login. If not → create new user with a random password and default placeholders
- 6. Returns JWT same as traditional flow

8.2 Role-Based Access Control (RBAC)

ROLE	PERMISSIONS
user	View dashboard, run algorithms, use sandbox, view content, like/comment blogs, take challenges
admin	All user permissions + Create/Edit/Delete algorithms, docs, blogs, news, courses, challenges, playground items. View analytics. Manage users. Upload images.

⚠️ Admin Creation

Admin accounts can only be created via the database seed script or by another admin promoting a user through the Admin Users Manager. The registration endpoint always creates users with `role: "user"`.

9. Deployment Guide

9.1 Local Development Setup

Prerequisites

- **Node.js** v18+ (LTS recommended)
- **Python** 3.9+ (with pip)
- **Git** (for version control)

Step-by-Step

```
# 1. Clone the repository
git clone https://github.com/your-repo/quantum-sim-lab.git
cd quantum-sim-lab

# 2. Install frontend dependencies
npm install

# 3. Install backend dependencies
cd backend
npm install

# 4. Install Python dependencies
pip install -r requirements.txt

# 5. Configure environment variables
# Copy .env.example to .env and fill in your values (see Section 10)

# 6. Start the application (from root directory)
cd ..
npm run dev
# This runs BOTH frontend (port 5173) and backend (port 8000) concurrently
```

9.2 Production Deployment (Vercel + Render)

Frontend → Vercel

1. Push code to GitHub
2. Go to vercel.com → Import Git Repository
3. Framework Preset: **Vite**
4. Build Command: `npm run build` (auto-detected)
5. Output Directory: `dist` (auto-detected)
6. Add Environment Variables:
 - `VITE_API_BASE_URL` = `https://your-backend.onrender.com/api`
 - `VITE_GOOGLE_CLIENT_ID` = Your Google OAuth Client ID
7. Deploy. The `vercel.json` file handles SPA rewrites automatically.

⚠ Vite Environment Variables

Vite **bakes** `VITE_*` environment variables into the JavaScript bundle at build time. Any change to these variables requires a **redployment** to take effect. They are NOT read at runtime.

Backend → Render

1. Go to render.com → New Web Service
2. Connect your GitHub repo

3. Root Directory: `backend`
4. Build Command: `npm install`
5. Start Command: `node server.js`
6. Add ALL environment variables from your `.env` (see Section 10)
7. Set `NODE_ENV=production`
8. Set `FRONTEND_URL` = Your Vercel URL (e.g., `https://sagqsim.vercel.app`)

⚠️ Render Free Tier — Cold Start

On Render's free tier, the backend service spins down after 15 minutes of inactivity. The first request after inactivity takes 30-60 seconds to "cold start." For production use, upgrade to the paid tier (\$7/month) to keep the service always running.

9.3 Cloudinary Configuration

Cloudinary is used in production for persistent image storage. In development, images are stored locally in `backend/uploads/`.

1. Sign up at cloudinary.com
2. Get your **Cloud Name**, **API Key**, and **API Secret** from the Dashboard
3. Set them in the backend environment variables
4. Images are automatically uploaded to the `quantum-sim-lab` folder with auto quality and format optimization

9.4 Google OAuth Setup

1. Go to [Google Cloud Console](https://console.cloud.google.com)
2. Create/select a project
3. Navigate to **APIs & Services** → **OAuth consent screen** → Configure → Set User Type to "External" → Publish the app
4. Navigate to **APIs & Services** → **Credentials** → Create OAuth Client ID → Web Application
5. Add **Authorized JavaScript Origins**:
 - `http://localhost:5173` (development)
 - `https://sagqsim.vercel.app` (production)
6. Copy the Client ID and set it as `GOOGLE_CLIENT_ID` (backend) and `VITE_GOOGLE_CLIENT_ID` (frontend)

9.5 GNews API Integration

1. Sign up at gnews.io
2. Get your API key from the dashboard
3. Set `GNEWS_API_KEY` in the backend environment
4. The backend proxies GNews requests to keep the API key server-side
5. An in-memory cache (15 min TTL) minimizes API calls

9.6 Redis Cache (Optional)

Redis is optional and disabled by default. If `REDIS_URL` is not set, all cache middleware is silently bypassed. To enable:

1. Provision a Redis instance (e.g., Redis Cloud, Upstash, or Render Redis)
2. Set `REDIS_URL` to the full connection string (e.g., `redis://default:password@host:port`)
3. Cache is used for docs, blogs, and news endpoints with automatic invalidation on CRUD operations

10. Environment Variables Reference

10.1 Root .env File (All Secrets)

VARIABLE	REQUIRED	DESCRIPTION	EXAMPLE
MONGO_URI	Required	MongoDB Atlas connection string	mongodb+srv://user:pass@cluster.mongodb.net/QuantumSim
JWT_SECRET	Required	Secret key for JWT signing (256-bit hex)	dac85118f37ab09... (128 char hex)
NODE_ENV	Required	Environment mode	development or production
FRONTEND_URL	Required	Frontend URL for CORS whitelist	https://saggsim.vercel.app
PYTHON_PATH	Optional	Absolute path to Python executable	python or c:\python\python.exe
CLOUDINARY_CLOUD_NAME	Prod Only	Cloudinary cloud name	sag
CLOUDINARY_API_KEY	Prod Only	Cloudinary API key	941388319486432
CLOUDINARY_API_SECRET	Prod Only	Cloudinary API secret	qF10nrxPoBy0vveRn79UB78hFDA
GOOGLE_CLIENT_ID	Required	Google OAuth Client ID (backend verification)	878678...apps.googleusercontent.com
VITE_GOOGLE_CLIENT_ID	Required	Same Google Client ID (frontend — Vite bakes it)	Same as above
GNEWS_API_KEY	Optional	GNews API key for live news	5752e7f6b3b39905...
REDIS_URL	Optional	Redis connection URL	redis://default:pass@host:6379

10.2 .env.production (Vite Build-Time Variables)

VARIABLE	VALUE	NOTES
VITE_API_BASE_URL	https://quantum-sim-lab-backend.onrender.com/api	API base URL with /api suffix
VITE_API_SERVER_URL	https://quantum-sim-lab-backend.onrender.com	Server root (for image URLs)

11. Pricing & Cost Analysis

The project currently operates on **100% free tiers**. Below is a comprehensive breakdown of current usage and what paid upgrades would cost for longer-term production use.

11.1 Current Free Tier Usage

SERVICE	FREE TIER LIMITS	CURRENT USAGE	RISK OF EXCEEDING
Vercel (Frontend)	100 GB bandwidth/month, 6000 build mins/month, 1 project, hobby use only	Low	Low Risk
Render (Backend)	750 free instance hours/month, 512 MB RAM, spins down after 15 min inactivity	Moderate	Medium Risk (cold starts)
MongoDB Atlas	512 MB storage, shared cluster (M0), 100 connections	Low	Low Risk
Cloudinary	25 credits/month (~25,000 transformations or 25 GB bandwidth)	Low	Low Risk
GNews API	100 requests/day, 10 articles per request	Low (cached 15 min)	Low Risk
Google OAuth	Free (unlimited)	—	No Risk

11.2 Paid Tier Pricing (Long-Term Production)

SERVICE	FREE TIER	RECOMMENDED PAID TIER	MONTHLY COST	KEY BENEFITS
Vercel	Hobby (Free)	Pro	\$20/month per team member	1 TB bandwidth, 24,000 build mins, commercial use allowed, team features, analytics, password protection
Render	Free	Starter	\$7/month	No cold starts (always on), 512 MB RAM, custom domains, better SLA
Render	—	Standard (if more load)	\$25/month	2 GB RAM, auto-scaling, more compute power
MongoDB Atlas	M0 (Free)	M10 (Dedicated)	\$57/month	10 GB storage, dedicated cluster, backups, performance advisor, monitoring
MongoDB Atlas	—	Serverless	~\$0.10/million reads	Pay-per-use, auto-scales, good for variable traffic
Cloudinary	Free (25 credits)	Plus	\$89/month	225 credits, more bandwidth, advanced transformations, video support
GNews API	Free (100 req/day)	Basic	\$17.99/month	10,000 req/day, 100 articles per request, historical search
GNews API	—	Professional	\$59.99/month	30,000 req/day, full archive access
Redis (Upstash)	Free (10,000 commands/day)	Pay-as-you-go	\$0.20 per 100K commands	Serverless Redis, auto-scale, no management
Custom Domain	—	.com/.in domain	\$10-15/year	Professional branding, SSL included on Vercel

11.3 Recommended Budget for Production

SCENARIO	SERVICES	MONTHLY COST
Minimum Viable Production	Vercel Free + Render Starter (\$7) + Atlas Free + Cloudinary Free + GNews Free	~\$7/month
Recommended Production	Vercel Pro (\$20) + Render Starter (\$7) + Atlas M10 (\$57) + Cloudinary Free + GNews Basic (\$18)	~\$102/month
Full Scale Production	Vercel Pro (\$20) + Render Standard (\$25) + Atlas M10 (\$57) + Cloudinary Plus (\$89) + GNews Pro (\$60) + Redis (\$5) + Domain (\$1)	~\$257/month

12. Server Migration Guide

If you need to move the application to a different server or hosting provider, follow this comprehensive guide.

12.1 What Needs to Change

ITEM	WHERE TO CHANGE	DETAILS
Backend URL	<code>.env.production</code> (frontend) + Vercel env vars	Update <code>VITE_API_BASE_URL</code> and <code>VITE_API_SERVER_URL</code> to new backend URL. Redeploy frontend after change.
Frontend URL	<code>FRONTEND_URL</code> in backend env	Update to new frontend URL for CORS whitelist
Google OAuth Origins	Google Cloud Console → Credentials	Add new frontend URL to "Authorized JavaScript Origins"
MongoDB Connection	<code>MONGO_URI</code> in backend env	If changing DB cluster, update connection string. If keeping same Atlas cluster, no change needed.
Cloudinary	No change needed	Cloudinary is a CDN — works from any server. Just set the same credentials.
Redis	<code>REDIS_URL</code> in backend env	If using a different Redis instance, update the URL
Python Environment	New server	Install Python 3.9+ and all packages from <code>requirements.txt</code> . Update <code>PYTHON_PATH</code> if needed.
Notebooks	Copy <code>backend/notebooks/</code>	All <code>.ipynb</code> files must be present on the new server

12.2 Migration to AWS / Azure / GCP

For enterprise-grade hosting:

- **Frontend:** Deploy to AWS S3 + CloudFront, Azure Static Web Apps, or GCP Cloud Storage + CDN
- **Backend:** Deploy to AWS EC2/ECS/Lambda, Azure App Service, or GCP Cloud Run
- **Database:** Use MongoDB Atlas (works with all clouds) or migrate to AWS DocumentDB / Azure CosmosDB
- **Python:** Ensure the server has Python installed. For containerized deployments, use a Docker image with both Node.js and Python

12.3 Docker Deployment (Recommended for New Servers)

A Docker-based deployment is recommended for consistent environments. You would need a `Dockerfile` with:

```
# Multi-stage: Node.js + Python
FROM node:18-slim
RUN apt-get update && apt-get install -y python3 python3-pip
WORKDIR /app/backend
COPY backend/package*.json ./
RUN npm install --production
COPY backend/ .
RUN pip3 install -r requirements.txt
EXPOSE 8000
CMD ["node", "server.js"]
```

12.4 Migration Checklist

1. ☐ Set up new server with Node.js 18+ and Python 3.9+
 2. ☐ Clone the repository to the new server
 3. ☐ Install all dependencies (npm install + pip install)
 4. ☐ Copy the .env file with updated URLs
 5. ☐ Update FRONTEND_URL in the backend environment
 6. ☐ Update VITE_API_BASE_URL in the frontend environment
 7. ☐ Redeploy frontend (if on Vercel, change env vars and redeploy)
 8. ☐ Update Google Cloud Console → Credentials → Authorized Origins
 9. ☐ Test Google Login, algorithm execution, image upload, and all features
 10. ☐ Wait 2–5 minutes for Google OAuth changes to propagate
-

13. Scaling & Performance

13.1 Current Performance Characteristics

Metric	Value	Notes
Frontend bundle size	~2.5 MB (gzipped)	Served via Vercel CDN with edge caching
API response time (Render)	50–200ms (hot), 30–60s (cold start)	Cold start eliminated on paid tier
Algorithm execution time	5–120 seconds	Depends on algorithm complexity. 120s timeout.
Rate limit	200 req/15 min global	Auth: 15 req/15 min in production
Max upload size	20 MB	Images only
JSON body limit	10 MB	For large blog/doc content

13.2 Caching Strategy

- **Redis Cache:** Optional middleware for docs, blogs, news endpoints. TTL: 1 hour. Automatically invalidated on CRUD operations via `invalidateCache(keyPrefix)`.
- **GNews Cache:** In-memory `Map` with 15-minute TTL. Minimizes external API calls to stay within the 100 req/day free tier.
- **Frontend:** `AlgorithmContext` caches the algorithm list in React state to avoid redundant API calls across page navigations.
- **Vercel CDN:** Static assets are served from Vercel's global CDN with automatic edge caching.

13.3 Scaling Recommendations

- **Horizontal Scaling:** The backend is stateless (no session storage). It can be horizontally scaled behind a load balancer.
- **Python Execution Bottleneck:** The quantum execution engine uses `child_process.exec()`, which is blocking. For high concurrency, consider offloading Python execution to a separate worker service (e.g., AWS Lambda, Cloud Functions, or a dedicated Python microservice with a job queue).
- **Database Indexing:** Key indexes are already in place: `Algorithm.id`, `User.username`, `User.email`, `PageVisit.path+date`.

14. Theming System

The application uses a robust CSS Variable-based theming system. Themes are toggled by changing the `data-theme` attribute on the `<html>` element.

Key CSS Variables

VARIABLE	PURPOSE	DARK MODE	LIGHT MODE
<code>--color-app-base</code>	Page background	Deep navy	White
<code>--color-app-surface</code>	Card backgrounds	Dark slate	Light gray
<code>--color-app-text-main</code>	Primary text	White	Dark navy
<code>--color-app-text-muted</code>	Secondary text	Gray	Slate
<code>--color-app-primary</code>	Accent color	Neon blue	Blue
<code>--color-app-border</code>	Borders	Dark border	Light border
<code>--color-app-glass</code>	Glassmorphism backgrounds	Transparent dark	Transparent light

Theming Rule

Never use hardcoded Tailwind colors like `bg-gray-900` or `text-blue-500`. Always use the CSS variables via `var(--color-app-*)` or the utility classes defined in `index.css` (e.g., `.app-glass`, `.app-card-hover`).

15. Quantum Execution Engine

15.1 How It Works

1. User selects an algorithm and configures parameters in the UI
2. Frontend sends `POST /api/run` with `{algorithmId, parameters}`
3. Backend maps frontend parameter names to Python variable names (e.g., "Number of Qubits" → `num_qubits`)
4. Backend constructs a `papermill` command:

```
python -m papermill "notebooks/grover-search.ipynb" "notebooks/output_TIMESTAMP.ipynb"
```



5. Papermill injects the parameters into the notebook's tagged "parameters" cell and executes all cells
6. Backend parses the executed notebook's output cells:
 - **Images** (base64 PNG): Circuit diagrams, histograms, Bloch sphere images
 - **Text** (stdout): Console output, measurement dictionaries
 - **Errors**: Stack traces (ANSI color codes stripped)
7. Backend extracts Bloch sphere angles (θ , ϕ) from text markers like `BLOCH_THETA=1.23`
8. Backend parses measurement dictionaries (e.g., `{ '101' : 500, '010' : 524 }`) into probability distributions
9. Results returned as JSON: `{circuit, graph, blochImage, console, measurements, blochSphere}`
10. Temporary output notebook is deleted after parsing

15.2 Available Quantum Algorithm Notebooks

#	ALGORITHM	NOTEBOOK FILE	PARAMETERS
1	Deutsch's Algorithm	deutsch.ipynb	Function Type
2	Deutsch-Jozsa Algorithm	deutsch-jozsa.ipynb	Number of Qubits, Function Type
3	Bernstein-Vazirani	bernstein-vazirani.ipynb	Secret String
4	Simon's Algorithm	simon.ipynb	Number of Qubits
5	Grover's Search	grover-search.ipynb	Number of Qubits, Target State, Iterations
6	Shor's Algorithm	shor.ipynb	Number to Factor, Attempts
7	Quantum Fourier Transform	qft.ipynb	Number of Qubits, Input State
8	Quantum Phase Estimation	quantum-phase-estimation.ipynb	Precision Qubits, Phase (θ)
9	Quantum Teleportation	quantum-teleportation.ipynb	Alpha (real), Beta (real)
10	Superdense Coding	superdense-coding.ipynb	—
11	BB84 Protocol	bb84.ipynb	Number of Qubits
12	VQE (Variational Quantum Eigensolver)	vqe.ipynb	Number of Qubits

16. Admin Panel

The Admin Panel is a comprehensive management dashboard accessible only to users with role: "admin" . It comprises 10 dedicated pages:

PAGE	ROUTE	FUNCTIONS
Admin Dashboard	/admin	Overview cards with navigation to all admin sections
User Analytics	/admin/user-analytics	OS distribution pie chart, Gender distribution, Age group histogram, Page views bar chart. Time filters: week/month/5 months/year/all
Users Manager	/admin/users	Paginated user list with search. Promote/Demote user roles (user ↔ admin)
News Manager	/admin/news	CRUD for internal news articles with Rich Text editor
Blogs Manager	/admin/blogs	CRUD for blog posts with Rich Text editor, image upload, category management
Docs Manager	/admin/docs	CRUD for documentation with section/subsection hierarchy, drag-and-drop reordering, Rich Text editor with KaTeX math
Courses Manager	/admin/courses	CRUD for courses with lecture management (title, YouTube video URL, duration)
Challenges Manager	/admin/playground	CRUD for circuit challenges. Configure qubits, target states, allowed gates
Add Algorithm	/admin/add-algorithm	Form to create new quantum algorithms with parameters
Edit Algorithm	/admin/edit-algorithm/:id	Edit existing algorithm details and parameters

17. Troubleshooting & Known Issues

ISSUE	CAUSE	SOLUTION
Google Login: "Origin not allowed"	Browser URL not in Google Console's Authorized Origins	Add the URL to Google Cloud Console → Credentials → OAuth Client → Authorized JavaScript Origins
Google Login: "invalid_client (401)"	VITE_GOOGLE_CLIENT_ID not set or wrong	Set correct Client ID in Vercel env vars and redploy
Google Login: "COOP block"	Browser blocking popup communication	Ensure vite.config.js has "Cross-Origin-Opener-Policy": "same-origin-allow-popups"
Google Login: Consent screen in Testing mode	Only test users can login	Google Console → OAuth consent screen → Publish App
Backend cold start (30-60s delay)	Render free tier spins down after inactivity	Upgrade to Render Starter (\$7/month) for always-on
Algorithm execution fails	Python/Qiskit not installed or wrong PYTHON_PATH	Install Python + requirements.txt. Set correct PYTHON_PATH
Images not loading in production	Cloudinary credentials not set	Set all 3 Cloudinary env vars and ensure NODE_ENV=production
CORS errors	Frontend URL not in backend's allowed origins	Set FRONTEND_URL in backend env to match exact frontend URL
Env vars not taking effect (Vercel)	Vite bakes vars at build time	Change env vars in Vercel dashboard and redploy
MongoDB connection error	IP not whitelisted in Atlas	Atlas → Network Access → Allow from Anywhere (0.0.0.0/0) for cloud hosting
Rate limit exceeded (429)	Too many requests from same IP	Wait 15 minutes or adjust limits in server.js

18. Future Roadmap

- **Containerization:** Dockerize the entire stack for consistent deployment across environments
 - **CI/CD Pipeline:** GitHub Actions for automated testing, building, and deployment
 - **Real Quantum Hardware:** Integration with IBM Quantum Experience API for execution on real quantum processors
 - **User Progress Tracking:** Track completed algorithms, challenge scores, and learning paths
 - **Collaboration Features:** Share notebooks, circuits, and results with other users
 - **API Rate Limiting per User:** Per-user API quotas instead of global IP-based limits
 - **WebSocket for Live Execution:** Stream algorithm execution progress in real-time
 - **Multi-language Support:** Internationalization (i18n) for Hindi and English
 - **Mobile App:** React Native companion app for on-the-go learning
 - **Email Notifications:** Blog comment notifications and algorithm completion emails
-

19. Appendix

A. SEO Configuration

The application includes comprehensive SEO meta tags in `index.html` :

- Primary meta tags (title, description, keywords, author, robots, theme-color)
- Open Graph (Facebook) meta tags for social sharing
- Twitter Card meta tags for Twitter sharing
- Google Fonts: Inter (UI), Outfit (headings), JetBrains Mono (code)

B. Fonts

Font	Usage	Weights
Inter	Primary UI text	300–900
Outfit	Headings and titles	300–900
JetBrains Mono	Code blocks and editor	400–700

C. Key NPM Scripts

Script	Command	Description
<code>npm run dev (root)</code>	<code>concurrently "vite" "npm run dev -- prefix backend"</code>	Runs both frontend and backend in parallel
<code>npm run build (root)</code>	<code>vite build</code>	Production build → <code>dist/</code>
<code>npm run dev (backend)</code>	<code>node --watch server.js</code>	Backend dev server with file watching
<code>npm start (backend)</code>	<code>node server.js</code>	Production backend server

D. Production URLs

Component	URL
Frontend (Vercel)	<code>https://sagqsim.vercel.app</code>
Backend API (Render)	<code>https://quantum-sim-lab-backend.onrender.com</code>
API Base	<code>https://quantum-sim-lab-backend.onrender.com/api</code>
Health Check	<code>https://quantum-sim-lab-backend.onrender.com/</code>
Cloudinary Images	<code>https://res.cloudinary.com/sag/image/upload/quantum-sim-lab/...</code>

Quantum Simulation Lab — Technical Project Documentation v1.0.0
Scientific Analysis Group (SAG), DRDO
Generated: 30 July 2026